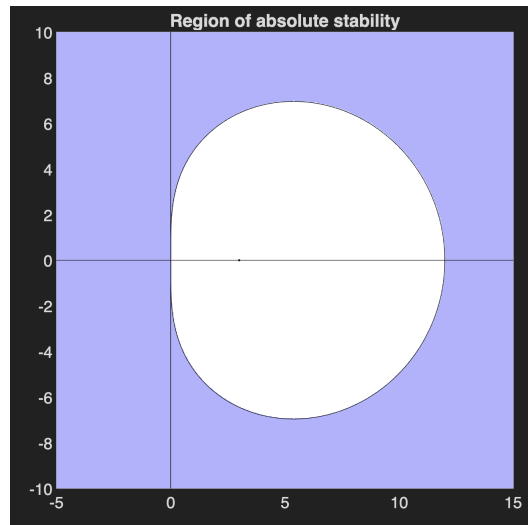


Math 502
Homework #5

Name: Gianluca Crescenzo

Exercise 8.1.

Solution. Below is the stability region for the TR-BDF2 method.



Exercise 8.4.

Solution. Using the test problem $u' = \lambda u$, we have:

$$u^* = u^n + \frac{z}{2} u^*.$$

Solving for u^* gives:

$$u^* = \frac{u^n}{1 - \frac{z}{2}}.$$

Hence:

$$\begin{aligned} u^{n+1} &= u^n + z u^* \\ &= u^n + \frac{z u^n}{1 - \frac{z}{2}} \\ &= \left(1 + \frac{z}{1 - \frac{z}{2}} \right) u^n. \end{aligned}$$

The stability function is then given by $R(z) = (1 + \frac{z}{1 - \frac{z}{2}})$. Expressing $\frac{z}{1 - \frac{z}{2}}$ as its Taylor series, we have:

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{4} + \frac{z^4}{8} + \dots$$

Compared to the Taylor expansion of e^z , note that $R(z)$ differs after the z^2 term. Hence the method is second order accurate.

Exercise 9.1.

Solution. (a) Manipulating our original scheme and re-indexing, we get:

$$\frac{u_i^{n+1} - u_i^{n-1}}{2k} = \frac{u_{i-1}^n - 2u_i^n + u_{i+1}^n}{h^2}.$$

Thus:

$$\tau = \frac{u(x_1, t_{n+1}) - u(x_1, t_{n-1})}{2k} - \frac{u(x_{i-1}, t_n) - 2u(x_1, t_n) + u(x_{i+1}, t_n)}{h^2}.$$

Taylor expanding yields:

$$\frac{u(x_1, t_{n+1}) - u(x_1, t_{n-1})}{2k} = u_t + \frac{k^2}{6} u_{ttt} + O(k^4).$$

Since $u_t = u_{xx}$, the right hand side becomes:

$$\frac{u(x_1, t_{n+1}) - u(x_1, t_{n-1})}{2k} = u_{xx} + \frac{k^2}{6} u_{xxxx} + O(k^4).$$

Hence:

$$\tau = (u_t - u_{xx}) + \frac{k^2}{6} u_{ttt} - \frac{h^2}{12} u_{xxxx} + O(k^4) + O(h^2).$$

Thus the order of accuracy is $O(k^2)$ and $O(h^2)$.

(b) The eigenvalues satisfy $\frac{-4}{h^2} < \lambda_p < 0$. From the leapfrog scheme, we have:

$$u^{n+2} = u^n + 2k\lambda u^{n+1}.$$

Taking $r = \lambda k$, the scheme can be rewritten as $z^2 - 2rz - 1 = 0$. The roots of this polynomial are $z = r \pm \sqrt{r^2 + 1}$. Note that the modulus of z is strictly greater than one, hence it is not stable.

Exercise 9.2.

Solution. (a) After adjusting `heat_CN.m` the following code outputs an error table and log-log plot using `error_table.m` and `error_loglog.m`.

```

1 mvals = [10 20 40 80 160];
2 ntest = length(mvals);
3
4 hvals = zeros(ntest,1);
5 E      = zeros(ntest,1);
6
7 for j = 1:ntest
8     [hvals(j),~,E(j)] = heat_CN(mvals(j));
9 end
10
11 error_table(hvals, E);
12 error_loglog(hvals, E);

```

```
>> hw6

WARNING *** k does not divide tfinal, k = 3.63636e-01
at time t = 1.09091e+00 max error = 2.52504e-03

WARNING *** k does not divide tfinal, k = 1.90476e-01
at time t = 9.52381e-01 max error = 9.07877e-04

WARNING *** k does not divide tfinal, k = 9.75610e-02
at time t = 9.75610e-01 max error = 1.53948e-04

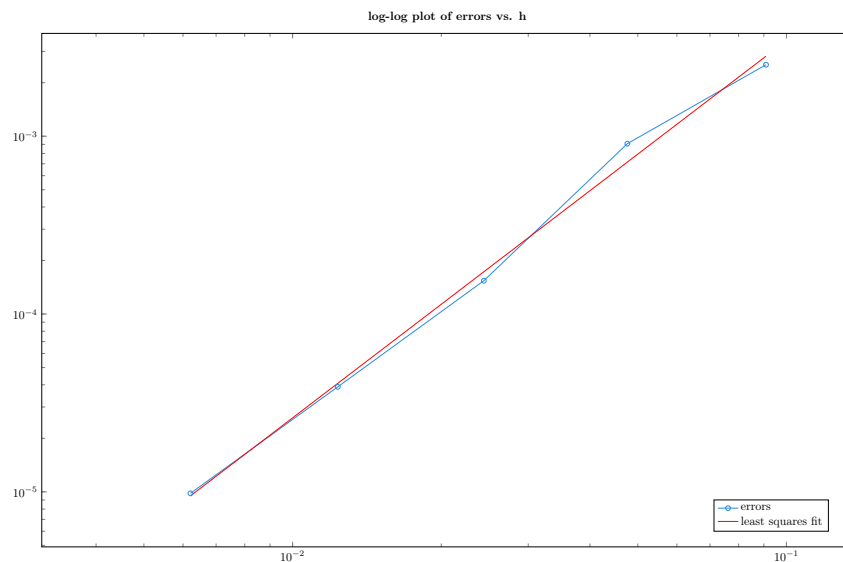
WARNING *** k does not divide tfinal, k = 4.93827e-02
at time t = 9.87654e-01 max error = 3.89510e-05

WARNING *** k does not divide tfinal, k = 2.48447e-02
at time t = 9.93789e-01 max error = 9.80754e-06

    h      error      ratio      observed order
0.09091  2.52504e-03      NaN          NaN
0.04762  9.07877e-04    2.78125    1.58190
0.02439  1.53948e-04    5.89730    2.65226
0.01235  3.89510e-05    3.95235    2.01844
0.00621  9.80754e-06    3.97154    2.00763

Least squares fit gives E(h) = 0.457212 * h^2.12203

>>
```



(b) Below is the updated code using the Backward Euler method:

```

1 function [h,k,error] = heat_CN1(m)
2
3 clf % clear graphics
4 hold on % Put all plots on the same graph (comment out if
   desired)
5
6 ax = 0;
7 bx = 1;
8 kappa = .02; % heat conduction coefficient:
9 tfinal = 1; % final time
10
11 h = (bx-ax)/(m+1); % h = delta x
12 x = linspace(ax,bx,m+2)'; % note x(1)=0 and x(m+2)=1
13 % u(1)=g0 and u(m+2)=g1 are known from BC's
14 k = 24*h^2; % time step
15
16 nsteps = round(tfinal / k); % number of time steps
17 %nplot = 1; % plot solution every nplot time steps
18 % (set nplot=2 to plot every 2 time steps, etc.)
19 nplot = nsteps; % only plot at final time
20
21 if abs(k*nsteps - tfinal) > 1e-5
22 % The last step won't go exactly to tfinal.
23 disp(' ')
24 disp(sprintf('WARNING *** k does not divide tfinal, k = %9.5e',k))
25 disp(' ')
26 end
27
28
29 % true solution for comparison:
30 % For Gaussian initial conditions u(x,0) = exp(-beta * (x-0.4)^2)
31 beta = 150;
32 utrue = @(x,t) exp(-(x-0.4).^2 / (4*kappa*t + 1/beta)) / sqrt(4*beta*
   kappa*t+1);
33
34 % initial conditions:
35 u0 = utrue(x,0);
36
37
38 % Each time step we solve MOL system U' = AU + g using the
   Trapezoidal method
39
40 % set up matrices:
41 r = kappa* k/(h^2);
42 e = ones(m,1);
43 A = spdiags([e -2*e e], [-1 0 1], m, m);
44 %A1 = eye(m) - r * A;
45 A2 = eye(m) + r * A;
46
47
48 % initial data on fine grid for plotting:
49 xfine = linspace(ax,bx,1001);
50 ufine = utrue(xfine,0);
51
52 % initialize u and plot:
53 tn = 0;

```

```

54 u = u0;
55
56 plot(x,u,'b.-', xfine,ufine,'r')
57 legend('computed','true')
58 title('Initial data at time = 0')
59
60 input('Hit <return> to continue ');
61
62
63 % main time-stepping loop:
64
65 for n = 1:nsteps
66     tnp = tn + k;    % = t_{n+1}
67
68     % boundary values u(0,t) and u(1,t) at times tn and tnp:
69
70     g0n = u(1);
71     g1n = u(m+2);
72     g0np = utrue(ax,tnp);
73     g1np = utrue(bx,tnp);
74
75     % compute right hand side for linear system:
76     uint = u(2:(m+1));    % interior points (unknowns)
77     rhs = A2*uint;
78     % fix-up right hand side using BC's (i.e. add vector g to A2*
79     uint)
80     rhs(1) = rhs(1) + r*(g0n);
81     rhs(m) = rhs(m) + r*(g1n);
82
83     % solve linear system:
84     uint = rhs;
85
86     % augment with boundary values:
87     u = [g0np; uint; g1np];
88
89     % plot results at desired times:
90     if mod(n,nplot)==0 | n==nsteps
91         ufine = utrue(xfine,tnp);
92         plot(x,u,'b.-', xfine,ufine,'r')
93         title(sprintf('t = %9.5e after %4i time steps with %5i grid
94         points',...
95         tnp,n,m+2))
96         error = max(abs(u-utrace(x,tnp)));
97         disp(sprintf('at time t = %9.5e max error = %9.5e',tnp,
98         error))
99         if n<nsteps, input('Hit <return> to continue '); end;
100     end
101
102     tn = tnp;    % for next time step
103 end

```

Using the same error-analysis in part (a), we get the following output:

```

>> hw6partb

WARNING *** k does not divide tfinal, k = 1.98347e-01

Hit <return> to continue

```

```

at time t = 9.91736e-01 max error = 3.04307e-02

WARNING *** k does not divide tfinal, k = 5.44218e-02

Hit <return> to continue
at time t = 9.79592e-01 max error = 3.09354e-03

WARNING *** k does not divide tfinal, k = 1.42772e-02

Hit <return> to continue
at time t = 9.99405e-01 max error = 8.14505e-04

WARNING *** k does not divide tfinal, k = 3.65798e-03

Hit <return> to continue

at time t = 9.98628e-01 max error = 2.09904e-04

WARNING *** k does not divide tfinal, k = 9.25890e-04

Hit <return> to continue
at time t = 9.99961e-01 max error = 5.31161e-05

WARNING *** k does not divide tfinal, k = 2.32917e-04

Hit <return> to continue
at time t = 9.99913e-01 max error = 1.33685e-05

    h      error      ratio      observed order
0.09091  3.04307e-02      NaN      NaN
0.04762  3.09354e-03  9.83683  3.53547
0.02439  8.14505e-04  3.79807  1.99461
0.01235  2.09904e-04  3.88037  1.99145
0.00621  5.31161e-05  3.95179  2.00038
0.00312  1.33685e-05  3.97324  1.99929

Least squares fit gives E(h) = 3.70715 * h^2.20379

>>

```

Hence the method is $O(h^2)$ accurate.

(c) I was not getting an unstable output. Unsure of what I did wrong.

Exercise 9.3.

Solution. Below is the updated code.

```

1 function [h,k,error] = heat_CN(m)
2
3 clf % clear graphics
4 hold on % Put all plots on the same graph (comment out if
   desired)
5
6 ax = -1;
7 bx = 1;
8 kappa = .02; % heat conduction coefficient:
9 tfinal = 1; % final time
10

```

```

11 h = (bx-ax)/(m+1);           % h = delta x
12 x = linspace(ax,bx,m+2)';    % note x(1)=0 and x(m+2)=1
13                               % u(1)=g0 and u(m+2)=g1 are known from BC'
                               % s
14 k = 4*h;                     % time step
15
16 nsteps = round(tfinal / k);    % number of time steps
17 nplot = 1;                    % plot solution every nplot time steps
18                               % (set nplot=2 to plot every 2 time steps, etc.)
19 nplot = nsteps;               % only plot at final time
20
21 if abs(k*nsteps - tfinal) > 1e-5
22     % The last step won't go exactly to tfinal.
23     disp(' ')
24     disp(sprintf('WARNING *** k does not divide tfinal, k = %9.5e',k))
25     disp(' ')
26     end
27
28
29 % true solution for comparison:
30 utrue = @(x,t) 0.5 * erfc( x ./ sqrt(4*kappa*t) );
31
32 % initial conditions (consistent with utrue as t -> 0+):
33 u0 = double(x < 0);           % 1 if x<0, 0 if x>=0
34
35
36 % Each time step we solve MOL system U' = AU + g using the
    Trapezoidal method
37
38 % set up matrices:
39 r = (1/2) * kappa * k/(h^2);
40 e = ones(m,1);
41 A = spdiags([e -2*e e], [-1 0 1], m, m);
42 A1 = eye(m) - r * A;
43 A2 = eye(m) + r * A;
44
45
46 % initial data on fine grid for plotting:
47 xfine = linspace(ax,bx,1001);
48 ufine = utrue(xfine,0);
49
50 % initialize u and plot:
51 tn = 0;
52 u = u0;
53
54 plot(x,u,'b.-', xfine,ufine,'r')
55 legend('computed','true')
56 title('Initial data at time = 0')
57
58 input('Hit <return> to continue ');
59
60
61 % main time-stepping loop:
62
63 for n = 1:nsteps
64     tnp = tn + k;             % = t_{n+1}
65

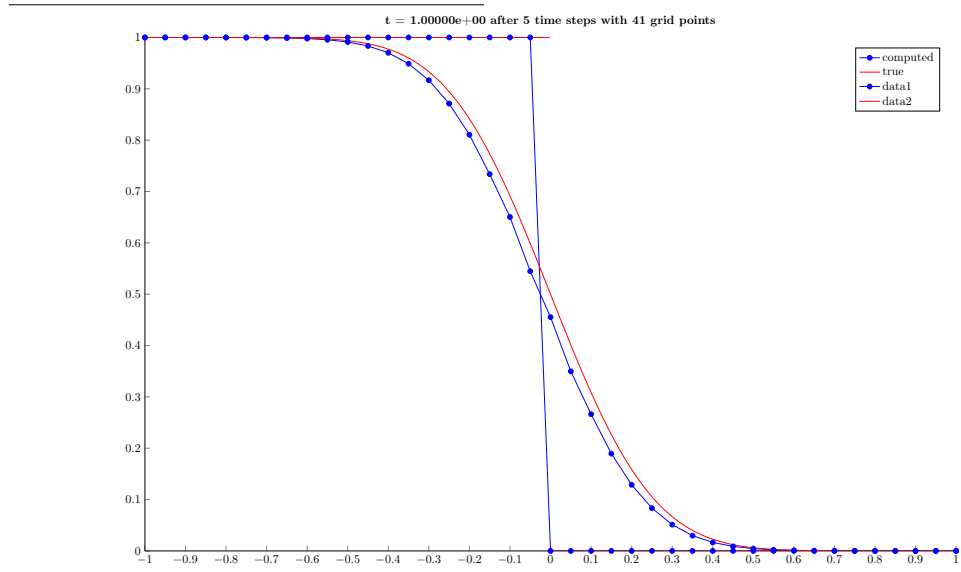
```

```

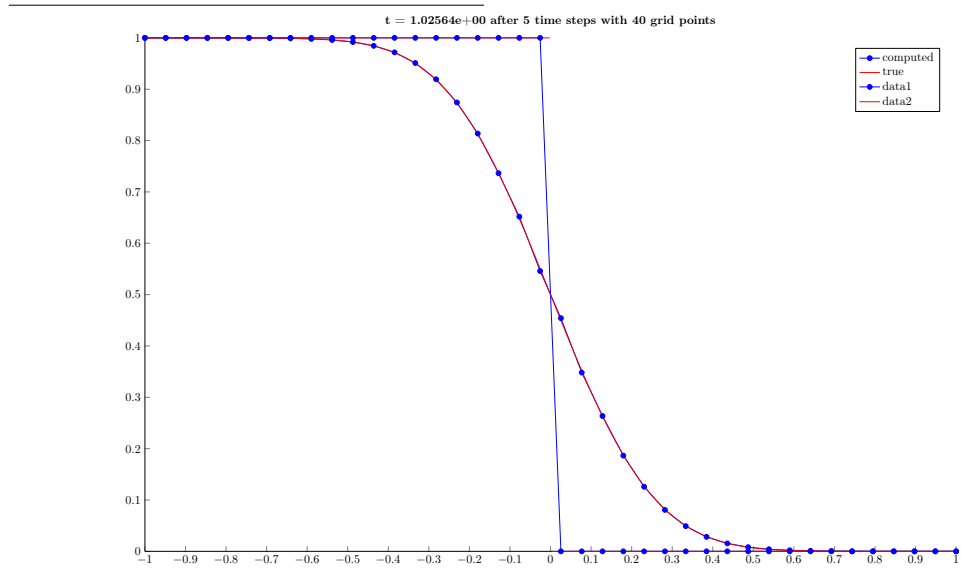
66 % boundary values u(0,t) and u(1,t) at times tn and tnp:
67
68 g0n = u(1);
69 g1n = u(m+2);
70 g0np = utrue(ax,tnp);
71 g1np = utrue(bx,tnp);
72
73 % compute right hand side for linear system:
74 uint = u(2:(m+1)); % interior points (unknowns)
75 rhs = A2*uint;
76 % fix-up right hand side using BC's (i.e. add vector g to A2*
    uint)
77 rhs(1) = rhs(1) + r*(g0n + g0np);
78 rhs(m) = rhs(m) + r*(g1n + g1np);
79
80 % solve linear system:
81 uint = A1\rhs;
82
83 % augment with boundary values:
84 u = [g0np; uint; g1np];
85
86 % plot results at desired times:
87 if mod(n,nplot)==0 | n==nsteps
88     ufine = utrue(xfine,tnp);
89     plot(x,u,'b.-',xfine,ufine,'r')
90     title(sprintf('t = %9.5e after %4i time steps with %5i grid
        points',...
            tnp,n,m+2))
91     error = max(abs(u-utrace(x,tnp)));
92     disp(sprintf('at time t = %9.5e max error = %9.5e',tnp,
        error))
93     if n<nsteps, input('Hit <return> to continue '); end;
94     end
95
96
97 tn = tnp; % for next time step
98 end

```

(a) Attached is the output when $m = 39$ and $k = 4h$.



(b) Attached is the output when $m = 38$ and $k = 4h$.



The better results for $m = 38$ are due to $x = 0$ lying between two grid points. The order of accuracy is first-order as the solution is not smooth at $t = 0$, reducing the usual second-order behavior of Crank-Nicolson method.